

Mixture-of-Expert with Expert Choice Routing

자연어 처리: 이관호(발표자), 황지현, 조해창, 박산희



- 1** **I n t r o d u c e**

- 2** **C o n v e n t i o n a l M o E**
 - **C o n c e p t s**
 - **L i m i t a t i o n s**

- 3** **M o E w i t h E x p e r t C h o i c e R o u t i n g**
 - **M e t h o d**
 - **R e s u l t**

- 4** **E x p e r i m e n t s**
 - **C o n c l u d e**

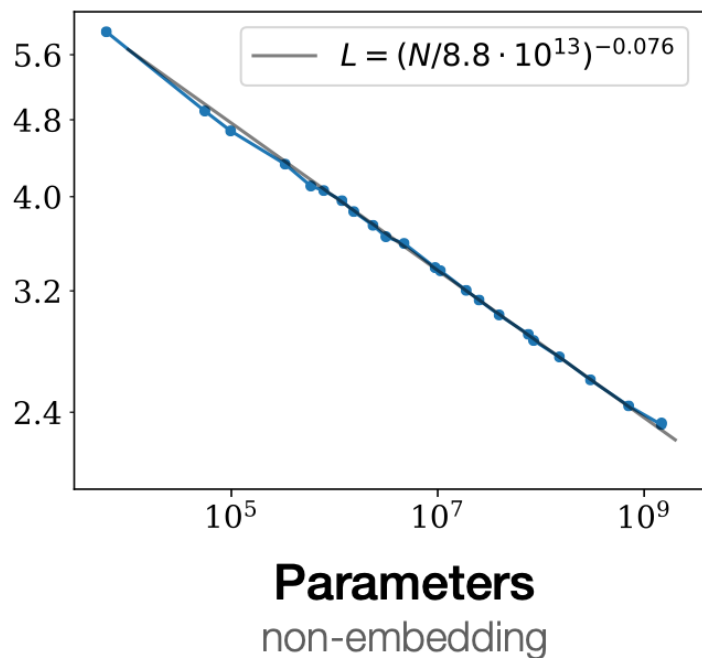
Part 1

Intro

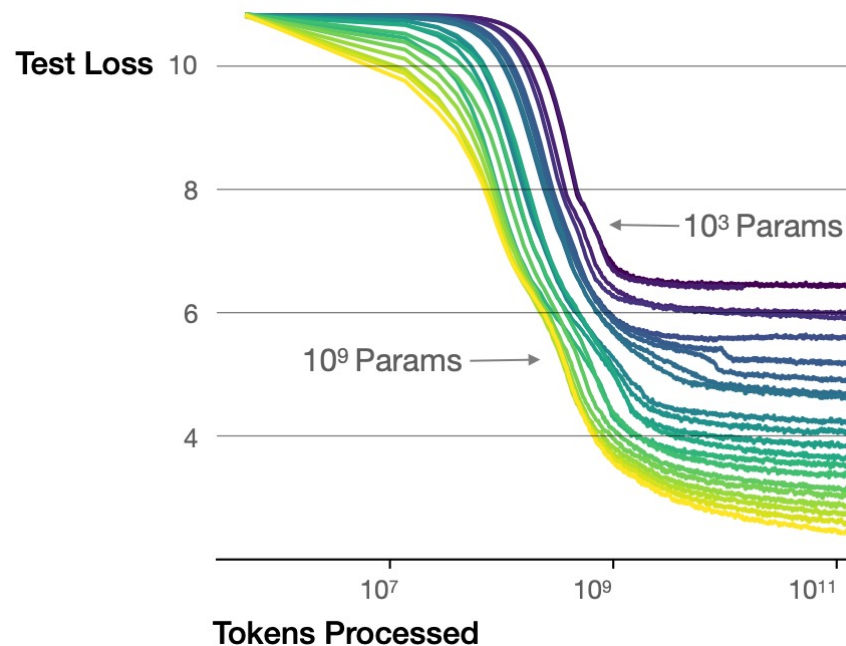
Introduce: MoE

Neural Scaling Laws (Kaplan et al., 2020)

- LLM의 성능은 데이터 사이즈의 크기, 모델의 파라미터 수가 증가에 비례한다



Model 크기와 Loss의 관계



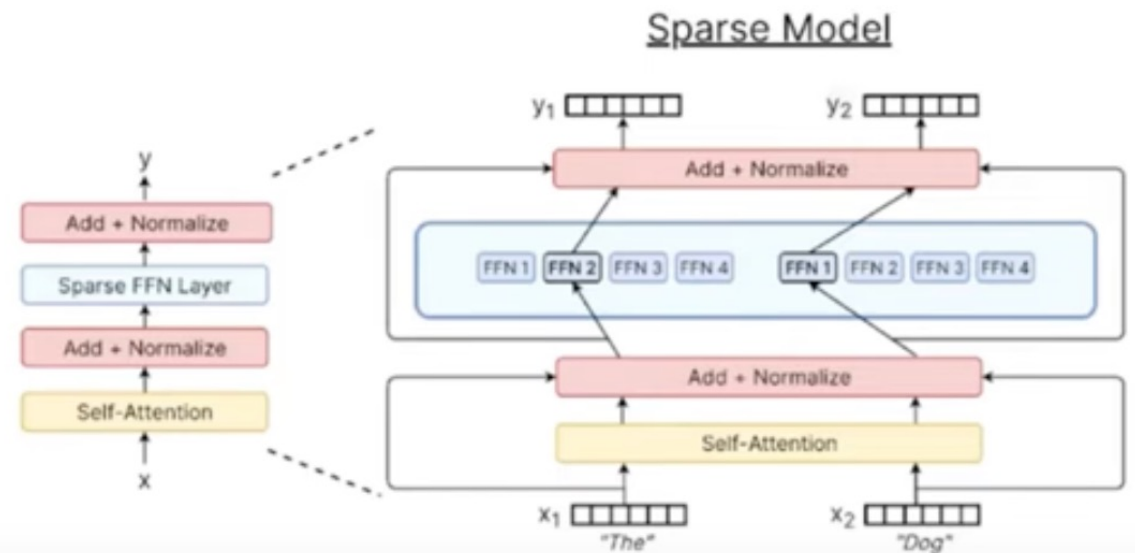
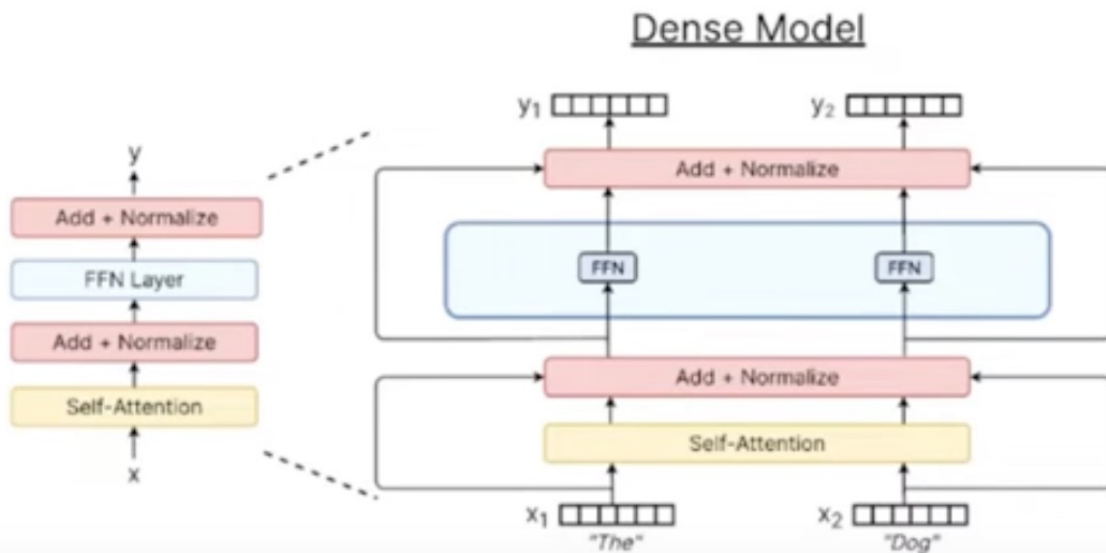
학습되는 토큰의 수에 따른 파라미터가 다른 모델의 성능 비교

Introduce: MOE

Dense Model VS Sparse Model

Sparse Model

- 각 인풋에 따라 전체 파라미터가 아닌 일부 파라미터만을 사용하자
- 인풋에 따라 달라지는 활성화 파라미터
- 인퍼런스시에 연산량이 유지되면서 전체 파라미터 수를 증가 시킬 수 있음



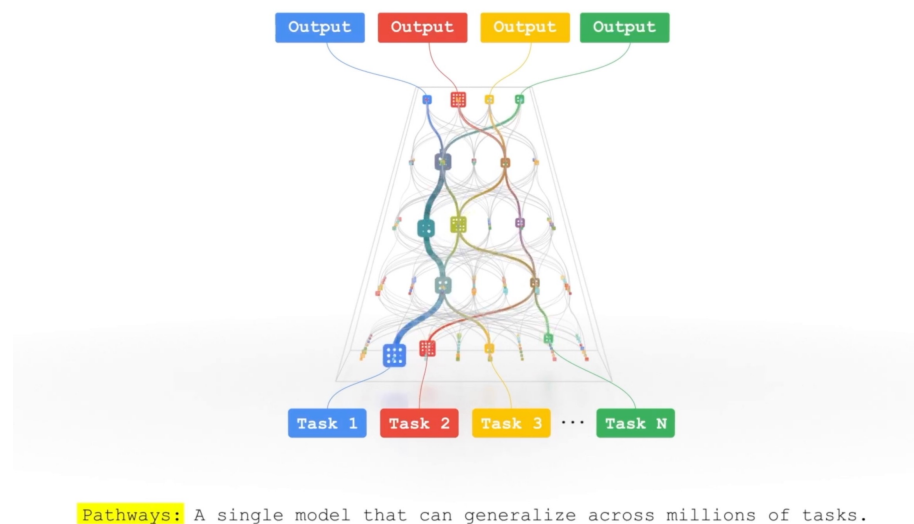
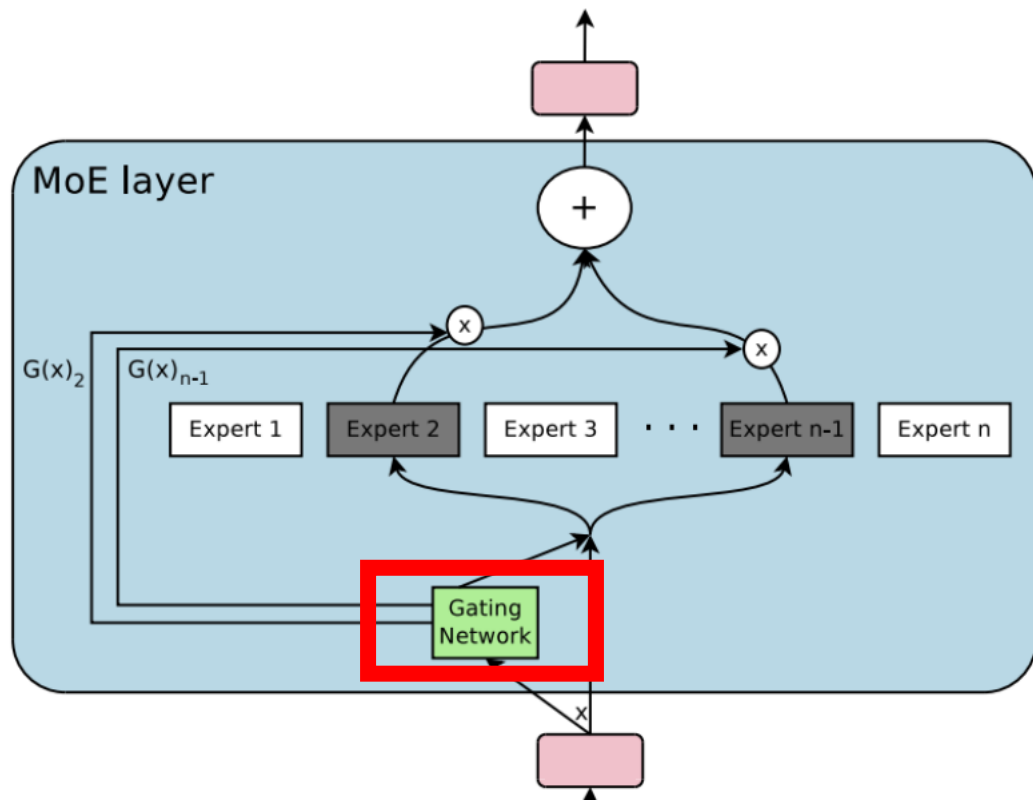
Part 2

Conventional MoE

Conventional MoE: Token Choice Experts

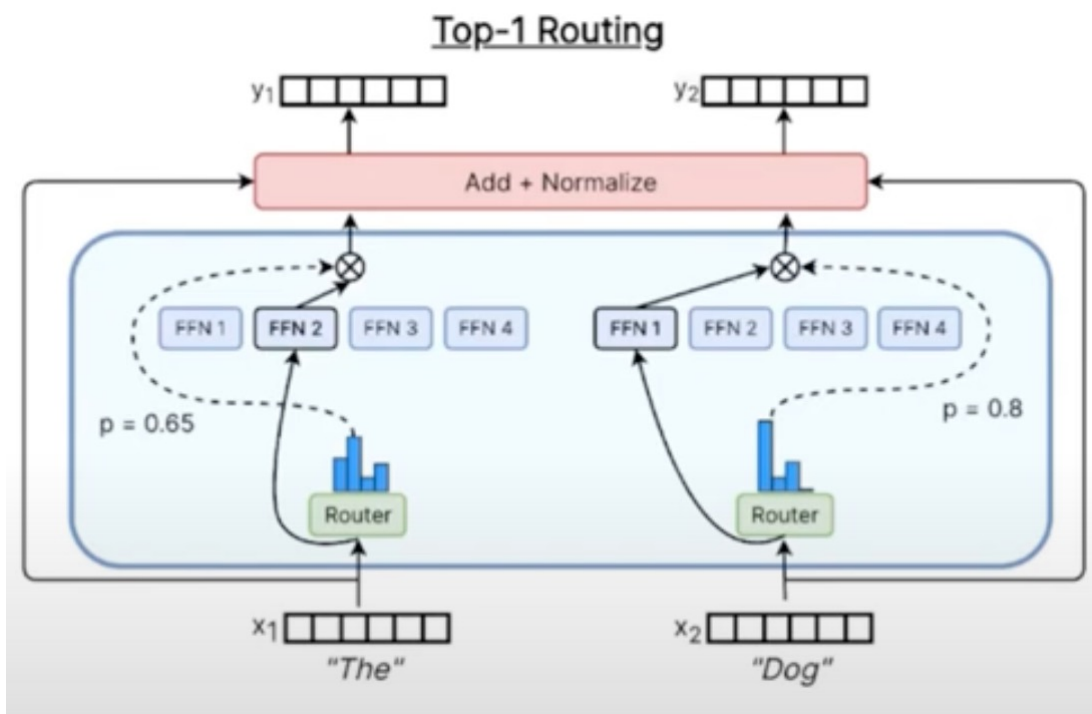
MoE

- Router : 각 인풋마다 어떤 Expert를 선택할지를 결정해주는 역할
- 의도: 전체 파라미터 수는 높이면서 활성화되고 있는 파라미터는 전체 파라미터 중 일부만 사용
- 전망: PaLM2에서 처음 등장한 “Pathways Language Model”에 대한 이해에 중요한 개념이 될 수 있음

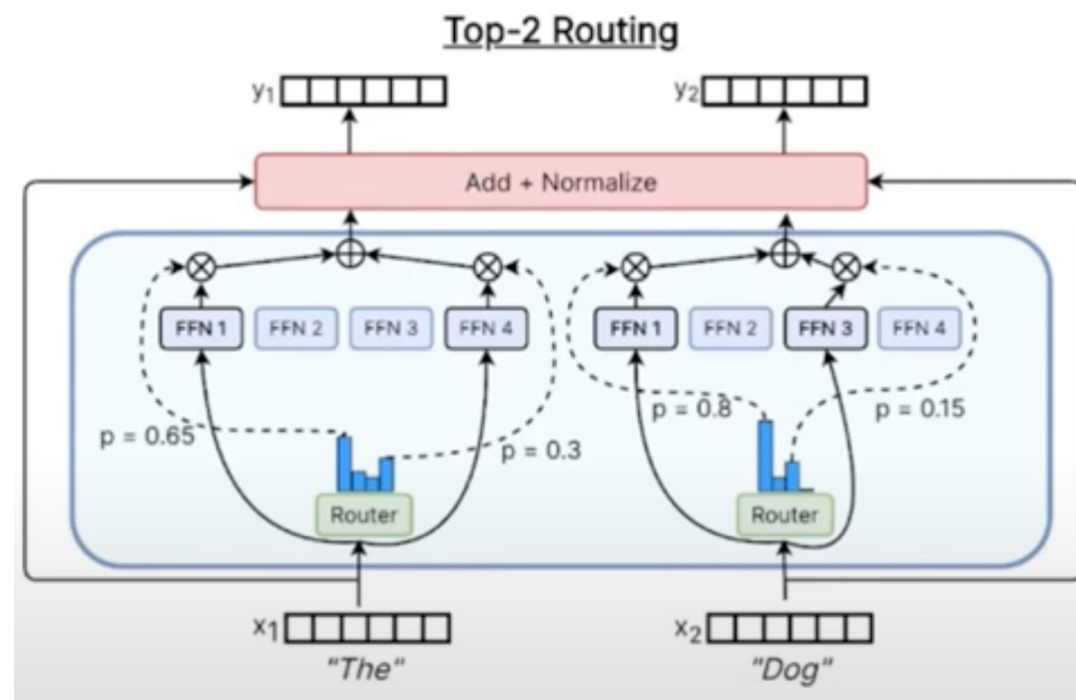


Conventional MOE : HISTORY

Switch Transformer
ST-Top1



Gshard
GS-Top2

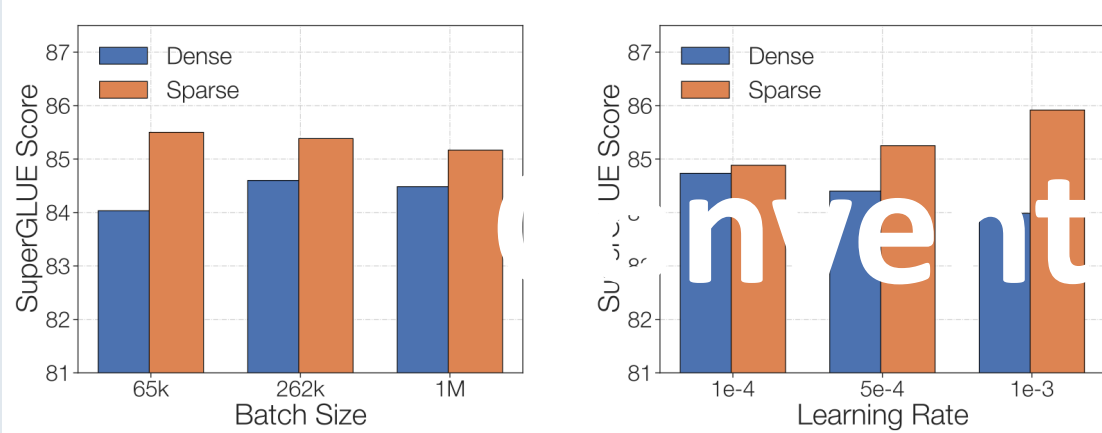


Conventional MOE : Limitations

주요 문제	상세 설명
Load Imbalance	• 몇몇 Experts만이 대부분의 토큰으로 학습
Under Specialization	• Input token이 gating을 통해 특정 Experts를 선택 • 선택을 많이 받지 못한 Experts는 학습이 충분히 되지 못함
Same Compute for Every Token	• 토큰당 k개의 Experts를 선택하여 computation량이 일정함

Conventional MOE : Mitigations

Conventional MOE의 해결 방안 : Different Hyper parameter, Auxiliary Loss



Different Hyper parameter

```
// (see routes, indices_list from forward pass).
// this loss is only for one layer. we add up the losses for
// each layer to get the total loss.
def switch_transformer_loss(model_loss, indices_list, routes,
    batch_size, num_tokens, num_experts, alpha=10e-2):

    // compute the fraction of tokens dispatched to expert i
    f = torch.vstack([torch.nn.functional.softmax(routes[i], dim=-1) for i in range(num_experts)]) / num_tokens

    // P is the probability fraction to expert i across *all tokens* in batch B
    P = torch.vstack([routes[i] for i in range(num_experts)]) / num_tokens

    auxiliary_loss = alpha * num_tokens * torch.dot(f, P)
    return model_loss + auxiliary_loss
```

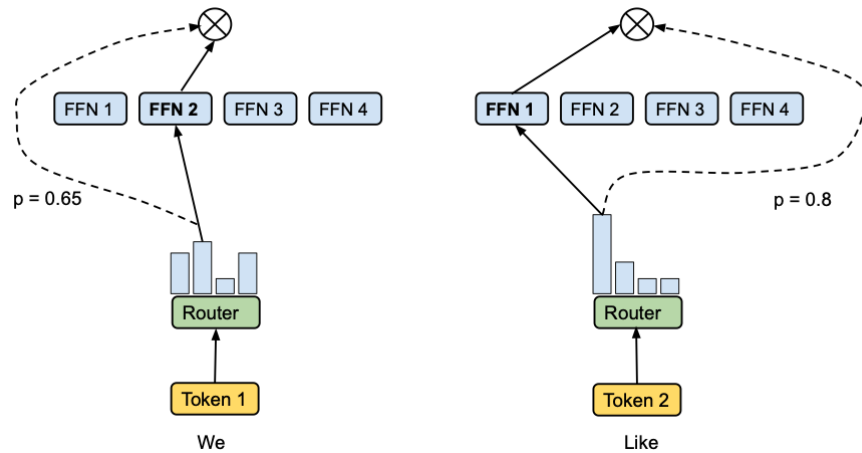
Auxiliary Loss
(보조 로스 함수)

Part 3

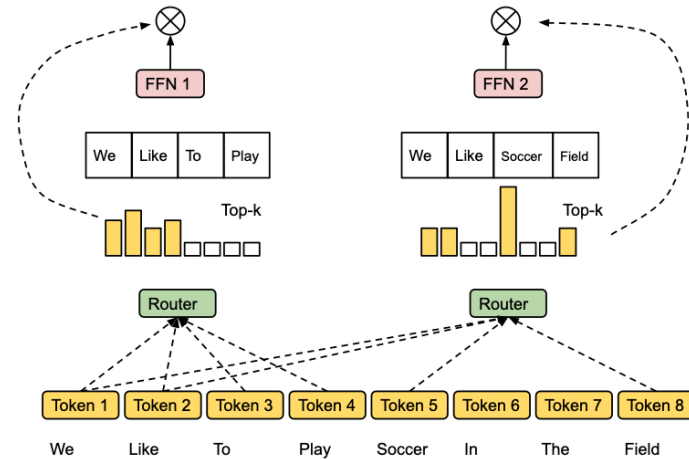
Expert Choice Routing

Experts Choice Routing

Conventional MOE Architecture VS Expert Choice Routing Architecture



Conventional MoE
(Tokens select one or two top-scoring experts)



Expert Choice Routing
(Each expert pick the top-k tokens)

d = last hidden states ,전체 토큰의 수 = 3, experts의 수 = 3, K=2 로 가정

$$S = \text{Softmax}(X \cdot W_g),$$

S

	e1	e2	e3
t1	0.2	0.6	0.2
t2	0.1	0.2	0.7
t3	0.5	0.4	0.1

= Softmax(

$X = n(\text{토큰의 수}) \times d$

x

$W = d \times e(\text{experts 수})$
(experts embeddings)

)

d = last hidden states ,전체 토큰의 수 = 3, experts의 수 = 3, K=2 로 가정

$$S$$

	e1	e2	e3
t1	0.2	0.6	0.2
t2	0.1	0.2	0.7
t3	0.5	0.4	0.1

= Softmax(

$X = n(\text{토큰의 수}) \times d$

$\times$

$W = d \times e(\text{experts 수})$
(experts embeddings)

)

$G, I = \text{TopK}(S^T, k)$

0.2	0.5
0.6	0.4
0.2	0.7

G

,

1	0	1
1	0	1
1	1	0

I



TopK

	t1	t2	t3
e1	0.2	0.1	0.5
e2	0.6	0.2	0.4
e3	0.2	0.7	0.1

S^T

d = last hidden states ,전체 토큰의 수 = 3, experts의 수 = 3, K=2 로 가정

	e1	e2	e3
t1	0.2	0.6	0.2
t2	0.1	0.2	0.7
t3	0.5	0.4	0.1

=

$$X = n(\text{토큰의 수}) \times d$$

x

$$W = d \times e(\text{experts 수})$$

(experts embeddings)

S

0.2	0.5
0.6	0.4
0.2	0.7

G

1	0	1
1	0	1
1	1	0

I



TopK

	t1	t2	t3
e1	0.2	0.1	0.5
e2	0.6	0.2	0.4
e3	0.2	0.7	0.1

S^T

$P = \text{Onehot}(I)$

1	0	0
0	0	0
0	0	1

1	0	0
0	0	0
0	0	1

1	0	0
0	1	0
0	0	0

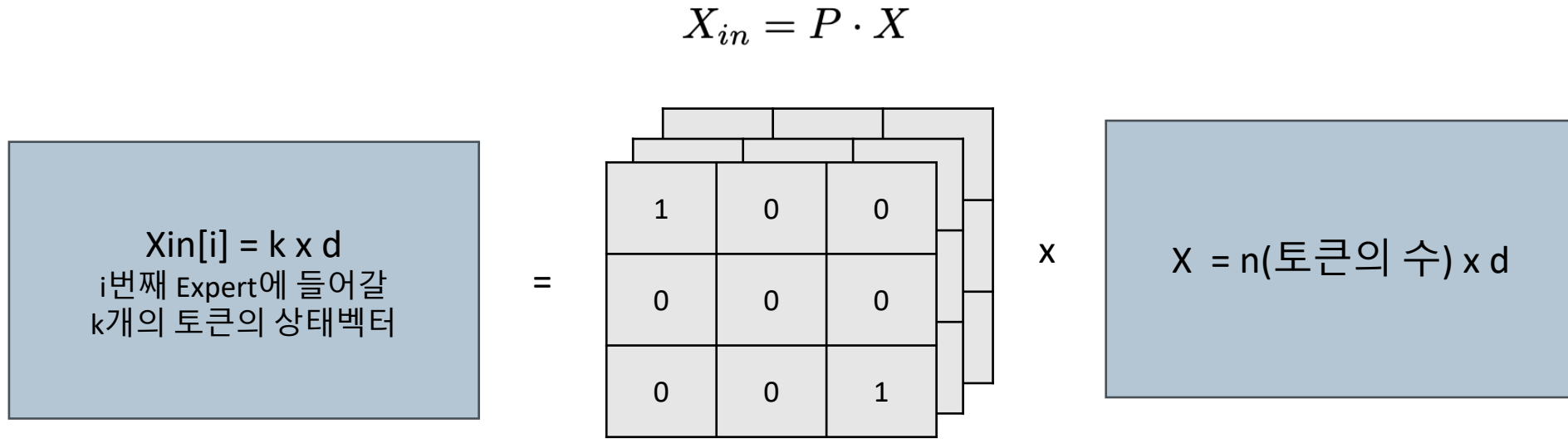


$$P$$

1	0	0
0	0	0
0	0	1

15

d = last hidden states ,전체 토큰의 수 = 3, experts의 수 = 3, $K=2$ 로 가정

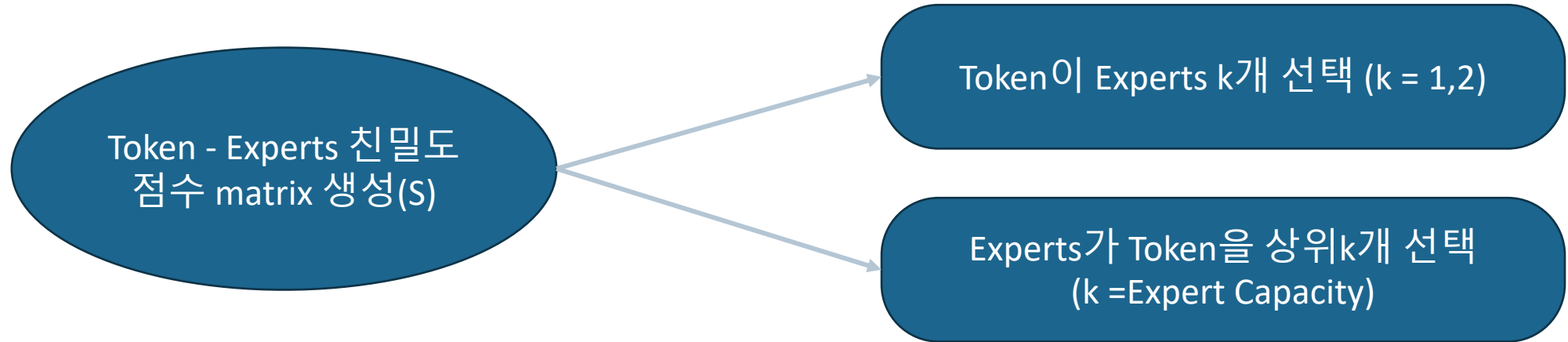


$$\forall i : X_e[i] = \text{GeLU}(X_{in}[i] \cdot W_1[i]) \cdot W_2[i]^T \quad (W_1, W_2 \text{는 } i\text{번째 Expert의 파라미터})$$

$$X_{out}[l, d] = \sum_{i,j} P[i, j, l] G[i, j] X_e[i, j, d]$$

Experts Choice Routing

Conventional MOE VS Expert Choice Routing



$$\text{Expert Capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) \times \text{capacity factor}$$

“모든 Expert가 정해진 수(k)의 토큰을 통해 훈련되므로 Load Imbalance를 해결가능하다”
→ 토큰의 선택을 못 받는 Expert가 없어지게 됨

Part 4

Experiments

Experiments: Setup

- Experts의 수에 대한 scaling 효과를 보기 위해 experts의 parameters 100M로 제한하고, 다양하게 구성함.
- 0.1B /64E → 모델/Experts 수
- act-params : token 별로 activation 되는 parameter의 수

Model	Type	n_{params}	$n_{\text{act-params}}$	L	M	H	n_{heads}	d_{head}	E
0.1B	Dense	130M	130M	12	768	3,072	12	64	-
0.1B/16E	MoE	548M	145M						16
0.1B/32E	MoE	1.0B	145M						32
0.1B/64E	MoE	1.9B	145M						64
0.1B/128E	MoE	3.7B	145M						128
8B	Dense	8.7B	8.7B	32	4,096	16,384	32	128	-
8B/64E	MoE	143B	9.8B						64

Experiments: Setup

- **Dataset**

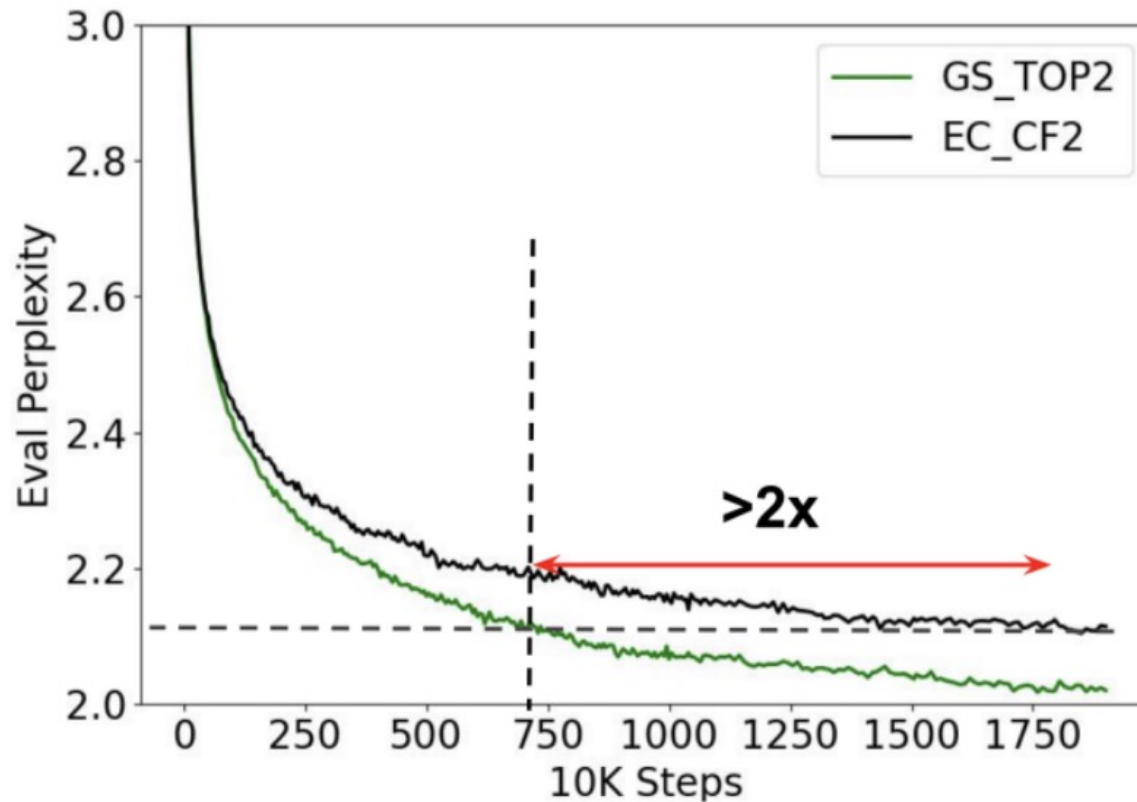
- **Train : GLAM** 1.6Trillion tokens in NLP (Filtered Webpages, Wikipedia, Conversations, Forums, Books, News)
- **Evaluation : GLUE, SuperGLUE**
 - selected 11 tasks (대규모 언어 모델을 위한 언어 이해 데이터셋)로 finetuning 평가

Corpus	Train	Dev	Test	Task	Metrics	Text Sources
BoolQ	9427	3270	3245	QA	acc.	Google queries, Wikipedia
CB	250	57	250	NLI	acc./F1	various
COPA	400	100	500	QA	acc.	blogs, photography encyclopedia
MultiRC	5100	953	1800	QA	F1 _a /EM	various
ReCoRD	101k	10k	10k	QA	F1/EM	news (CNN, Daily Mail)
RTE	2500	278	300	NLI	acc.	news, Wikipedia
WiC	6000	638	1400	WSD	acc.	WordNet, VerbNet, Wiktionary
WSC	554	104	146	coref.	acc.	fiction books

- **Model Training**

- 이전 연구들(i.e. GShard, Switch Transformers)에서 load balance를 위해 사용하던 auxiliary loss를 따로 사용하지 않고 학습.

Experiments: Training Efficiency



- Capacity factor of 2 (EC-CF2)모델과 GShard top-2 모델을 특정한 단계 동안 훈련을 진행하면서 수렴을 비교
- EC-CF2가 GShard top-2 모델과 비교할 때, 동일한 perplexity 에 도달하는 데 필요한 단계 수가 절반 이하이며, 각 단계는 약 20% 빠름
- GShard top-2의 딜레이는 load imbalance으로 인해 발생, 즉, 일부 experts 과도한 양의 토큰을 처리하면서 지연이 발생

Experiments: Capped Expert Choice

Method	Max # of Experts	Avg acc.
EC-CAP2	2	83.2 ± 0.4
EC-CAP3	3	84.0 ± 0.4
EC-CF2	-	84.0 ± 0.2
Hash Layer	-	81.3 ± 0.1

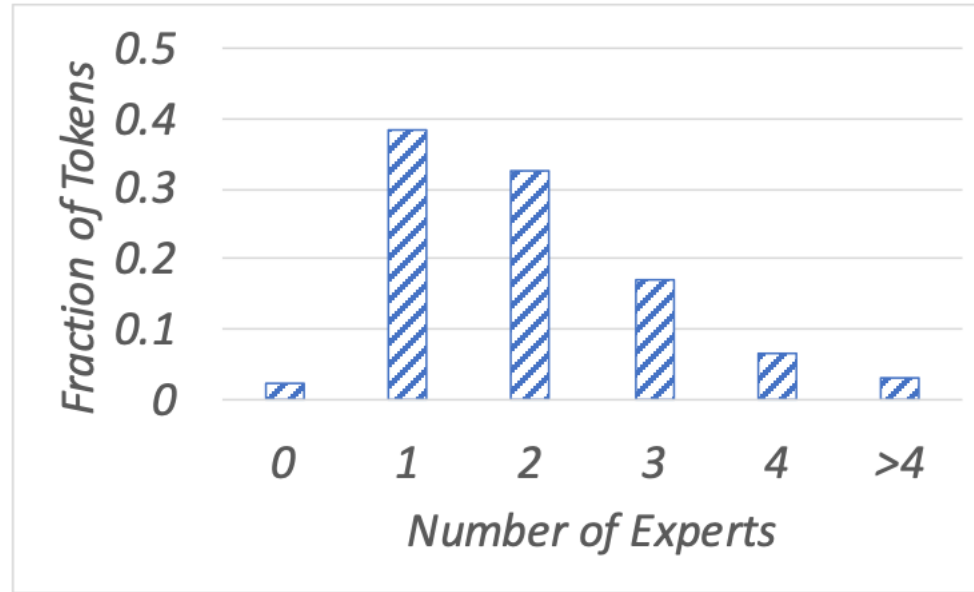
- EC-CAP2 와 EC-CAP3은 각 토큰의 전문가 수를 최대 2와 3으로 제한한 변형 모델을 의미하며, 이 모델을 써서 GLUE 및 SuperGLUE 벤치마크에서 선택한 11가지 Task의 평균 accuracy 을 측정
 - EC-CAP2 는 accuracy 0.8 포인트 저하
 - EC-CAP3는 상대적으로 기본모델 EC-CF2와 유사한 결과를 보여줌
- 토큰당 가변적인 전문가 수를 허용하는 것이 실제로 도움이 된다는 것을 확인

Experiments: Model Evaluation

Name	Metric	Split	100M/32E			8B/64E		
			ST Top-1	GS Top-2	EC-CF2	ST Top-1	GS Top-2	EC-CF2
BoolQ	acc	dev	74.5	79.0	79.3	89.1	89.5	89.2
CB	acc	dev	80.6	81.3	92.2	93.8	96.7	100
CoLA	acc	dev	87.5	92.2	93.8	88.3	87.5	89.1
MNLI	acc	dev	83.1	87.8	88.0	90.7	91.4	91.1
MRPC	acc	dev	82.3	85.2	84.4	89.3	91.7	90.6
QNLI	acc	dev	91.6	91.9	92.5	94.5	94.9	95.0
QQP	acc	dev	90.1	91.5	92.0	92.1	92.5	93.8
RTE	acc	dev	75.0	79.1	78.1	91.0	92.2	95.2
SST2	acc	dev	93.3	94.4	95.4	97.1	98.0	97.7
WiC	acc	dev	62.5	65.9	69.8	74.5	76.4	83.8
WNLI	acc	dev	65.6	64.1	68.8	78.1	82.8	92.8
Avg	-	-	80.6	83.5	85.0	88.9	90.3	92.6

- 향상된 perplexity가 직접적으로 downstream task 서 더 나은 성능으로 이어지는지 확인하기 위해 Finetuning 된 해당 모델을 사용하여 GLUE 및 SuperGLUE 벤치마크에서 선택한 11가지 Task 를 평가
- 실험에서 같은 조건에서 3가지 MoE 모델 비교를 수행하였음
 - Switch Transformer top-1 gating (ST Top-1), GShard top-2 gating (GS Top-2) and our method (EC-CF2)
 - 예) 100 M / 32 E : 100M의 파라미터 수와 32개의 Experts를 사용한 모델
- 실험 결과에서 EC-CF2가 다른 방법보다 거의 모든 지표에서 더 효과적

Experiments: Variable Experts per Token



- EC-CF2의 각 토큰에 대한 Number of Experts 비율
 - 23%의 토큰이 3,4개의 Experts를 선택
 - 4개 이상의 Experts를 선택한 토큰은 3%
- EC의 Flexible Expert Choice 장점확인 가능

Experiments: Conclusion

- Mixture-of-Experts(MoE) 모델을 위한 Expert Choice Routing이라는 새로운 라우팅 방법을 제안
 - 이 방법은 기존의 MoE 방법에서 나타나는 Load Imbalance와 Under Specialization 문제를 해결하며, 각 토큰에 대해 다양한 수의 전문가를 선택할 수 있게 합니다.
- Expert Choice Routing을 사용한 모델은 GShard 및 Switch Transformer 모델과 비교했을 때 Training Efficiency 에서 2배 이상 향상되었으며, GLUE 및 SuperGLUE 벤치마크에서 11개 항목에서도 대부분 더 높은 정확도를 보입니다.

감사합니다
